

Einfache Methoden – Signatur, Parameter, Rückgabewert

Pseudocode (IHK-Stil):

```
funktion summe(a: Integer, b: Integer) → Integer
    return a + b
end funktion
```

```
ergebnis = summe(3, 5) // 8
```

Java-Beispiel:

```
public class MethodenDemo {
    static int summe(int a, int b) {
        return a + b;
    }
    public static void main(String[] args) {
        int ergebnis = summe(3, 5);
        System.out.println(ergebnis); // 8
    }
}
```

Einfache Methoden – Erklärung & Beispiele

Eine Methode hat **Signatur** (Name, Parameter, Rückgabewert).
In Java: `Rückgabetyt name(Parameterliste)`.

Typische Fehlerquellen:

- Rückgabewert vergessen
- `void` bei Methoden ohne Rückgabewert
- Parameter-Reihenfolge falsch

Prüfungstipp: Methoden immer klar benennen, Signatur korrekt angeben!

Teil	Beispiel
Name	summe
Parameter	(int a, int b)
Rückgabewert	int
Aufruf	summe(3,5)

Methodenüberladung (Overloading)

Pseudocode (IHK-Stil):

```
funktion summe(a: Integer, b: Integer) → Integer
    return a + b
end funktion
```

```
funktion summe(a: Integer, b: Integer, c: Integer) → Integer
    return a + b + c
end funktion
```

Java-Beispiel:

```
public class OverloadingDemo {
    static int summe(int a, int b) { return a + b; }
    static int summe(int a, int b, int c) { return a + b + c; }

    public static void main(String[] args) {
        System.out.println(summe(2,3)); // 5
        System.out.println(summe(1,2,3)); // 6
    }
}
```

Methodenüberladung – Erklärung & Beispiele

Überladen = mehrere Methoden mit gleichem Namen, aber unterschiedlicher Parameterliste.

****Vorteile:****

- Einheitlicher Name für ähnliche Operationen
- Klarere Struktur

****Typische Fehlerquellen:****

- Nur Rückgabetyp ändern → nicht erlaubt!
- Verwechslung der Parameterreihenfolge

****Prüfungstipp:**** Overloading-Aufgaben kommen oft in Verbindung mit Mathe-Funktionen oder Konstruktoren.

Beispiel	Aufruf
summe(int,int)	summe(2,3) → 5
summe(int,int,int)	summe(1,2,3) → 6

Parameterübergabe (by Value vs. Referenz)

Pseudocode (IHK-Stil):

```
funktion addiereEins(x: Integer)
    x = x + 1
end funktion
```

```
funktion setzeName(k: Kunde)
    k.name = "Neuer Name"
end funktion
```

Java-Beispiel:

```
class Kunde { String name; }

public class ParameterDemo {
    static void addiereEins(int x) {
        x = x + 1;
    }
    static void setzeName(Kunde k) {
        k.name = "Neuer Name";
    }
    public static void main(String[] args) {
        int a = 5;
        addiereEins(a);
        System.out.println(a); // bleibt 5

        Kunde k = new Kunde();
        k.name = "Alt";
        setzeName(k);
        System.out.println(k.name); // Neuer Name
    }
}
```

Parameterübergabe – Erklärung & Beispiele

In Java werden Parameter ****immer by Value**** übergeben:

- Primitive Typen → Wertkopie → Änderungen wirken nur lokal
- Objektreferenzen → Kopie der Referenz → Änderungen am Objekt wirken global

****Typische Fehlerquellen:****

- Verwechslung "Call by Value" und "Call by Reference"
- Änderungen bei Objekten unterschätzt

****Prüfungstipp:**** IHK fragt oft: "Warum bleibt int unverändert, Objekt aber nicht?"

Typ	Verhalten
int (primitive)	bleibt unverändert
Objekt (Referenz)	Attribute ändern sich